National AI Hackathon 2026 by SchubKI

16-21 of October 2026 (16 – opening, 17-18 – work, 19-20 – evaluation, 21 - closing)

SereneDB

Challenge name

Making Sense of Logs: The Right Insights for the Right People

Challenge description

In this 48-hour challenge, build a working prototype that turns log data into useful answers for three roles. An engineer needs to investigate an error. A team lead needs to understand recurring problems. A manager needs to decide what to prioritise. Use SereneDB to find relevant evidence and calculate reliable summaries, then let an AI agent explain the results and recommend next steps. Apply your SQL, programming and data analysis skills to one clear workflow that works from question to evidence to recommendation.

Challenge (input + details)

An AI agent can help explain a failure, but millions of log lines will not fit into one prompt.

Your task is to give the agent a way to ask focused questions about the logs. The database searches and calculates; the agent uses the returned evidence to explain what happened. Start with the supplied BGL and OpenStack datasets, loader, schema and team-ownership table. These form the base corpus: the collection of logs used for the core challenge.

Aim for one complete workflow in 48 hours. Suggested timing: hours 0-6, inspect the data and run your first queries; 6-18, build search and summary tools; 18-30, connect the agent and support the three roles; 30-40, check answers against query results; 40-48, finish the README and rehearse the demo. Keep optional extensions until the core workflow works.

Three people ask about the same logs, but need different evidence and decisions.

Engineer. What failed, in which component, and what happened just before and after? Return the relevant error messages and nearby events, explain a possible cause, and suggest the next check.

Team lead. Which problems recur, how often, and which components or teams are involved? Use counts and trends over a stated period, with the supplied ownership table, to suggest what the team should investigate first.

Manager. Which observed problems deserve attention first, and what should happen next? Give a short, prioritized explanation of the risks and actions. Use measured frequency and affected components; do not invent financial losses, downtime or customer impact.

Use the same agent and base corpus for all three roles. Change the questions, evidence selected and recommended actions for each role; changing only the length of the answer is not enough.

What does the agent need from you.

Build a small set of tools: functions the agent can call to search logs, retrieve nearby events, and calculate summaries. Describe what each tool does, which inputs it accepts, and what it returns. The agent should be able to choose the appropriate tool from these descriptions. A small set of parameterized queries is sufficient. The agent must select and call the appropriate tools; open-ended SQL generation is not required.

Do the search and calculations in SereneDB before sending results to the agent. Search should return a limited set of relevant log entries; aggregation should return calculated counts or trends. Include the query and its filters, and indicate when results are truncated. A few retrieved examples cannot establish how often an error occurs in the full dataset.

For the engineer, implement full-text search with time filters and nearby events. For the team lead and manager, implement counts grouped by component or team and a trend over a suitable time period. Use the fields available in the supplied schema. Phrase matching, fuzzy matching and more detailed comparisons are optional improvements.

Optional: let the agent manage its own workspace.

After the core workflow works, you may expose tools that create an index for a selected time range and remove it after an investigation. An index helps the database find relevant records faster. Managing indexes through the agent is an optional extension, not a minimum requirement.

For the core submission, focus on correct queries using the provided instance. If you explore indexing, compare the time needed to answer repeated questions with and without the index. Explain the trade-off between the time spent creating it and the time saved on later queries.

Optional: explore larger datasets.

BGL and OpenStack are sufficient for the core challenge and the full main score. Once your workflow works, you may try the larger datasets already listed below, including remote Parquet data queried through SereneDB. Attempt this only within the available time and resources; no additional datasets, services or organiser-provided infrastructure are needed for the core submission.

Recommended: expose your tools through an MCP server.

MCP (Model Context Protocol) lets an agent discover and call your tools. Use the MCP SDKs to expose the same search and summary functions so Claude can use them. This avoids building a separate interface. A CLI, terminal UI or web UI is also acceptable if it demonstrates the agent calling tools. Use only the tools and resources listed in this brief. Using an existing agent to call your tools is sufficient; a custom agent application is not required.

Two things before you start.

First, connect the SereneDB docs MCP server to your existing coding assistant: Claude Code, Codex or Cursor. Use the supplied setup guide and SQL reference to get your first query working. SQL search and aggregation are enough for the core challenge; vector and hybrid search are optional.

Second, ground every measured claim in a query that actually ran. Keep the query and its result so the answer can be checked. Separate observed facts from possible explanations and recommended checks. If the logs do not establish a cause or impact, say what remains unknown. Count matching log entries accurately; do not automatically treat them as separate incidents.

What you get:

- A SereneDB instance
- Corpus links and schema
- The team-ownership table that maps components to the teams that own them
- Docs at serenedb.com/docs: SQL reference, full-text search operators, vector and hybrid search, remote table syntax
- A docs MCP server for Claude Code, Codex, Cursor. Setup guide at serenedb.com/docs

Minimal requirements

- An agent answers at least one question for each of the three roles. Each answer uses relevant evidence, an appropriate level of detail and a next step suited to that role.
- Uses both retrieval and aggregation on BGL and OpenStack. At least one answer uses full-text search. At least one uses an aggregation over the whole base corpus, such as counts by dataset or mapped team, rather than a sample of retrieved lines. State the data scope and any filters. Use both BGL and OpenStack, but they may support different parts of the workflow. The worked investigation may focus on one dataset. Whole-corpus counts may be calculated separately for each dataset and reported together; merging the datasets is not required.

- Gives practical recommendations supported by the findings. Explain what to check or fix first and why. A plausible next diagnostic step is acceptable when the available logs cannot confirm a root cause.
- Exposes search and summary capabilities as tools the agent can call. Tool descriptions explain their purpose, inputs and outputs clearly enough for the agent to select them.
- Shows its work. Include the executed queries and relevant results behind factual claims and numbers. Clearly label hypotheses and limitations; keep the evidence accessible for checking.
- Runs from a clean checkout with setup instructions, using the resources already listed in this brief. Include how to connect to the provided SereneDB instance, load the base data with the supplied loader and start the agent workflow.

Expected output

- A public repository with a README explaining the workflow, setup, how to run it and known limitations.
- A working prototype. An MCP server is recommended so judges can connect Claude using your instructions. A CLI, terminal UI or web UI is also acceptable. A separate graphical interface is not required.
- The tools you built, with their descriptions, accepted inputs and returned outputs. These can be documented in the README.
- One worked investigation of a real issue or recurring pattern in the base data. Show the questions, tool calls, executed queries and relevant results, then the answer and recommended action for each role. Include both search evidence and aggregation. State what the data does and does not establish.
- A presentation/demo of no more than 5 minutes showing the working workflow, the three role-specific answers and the query evidence behind them.
- Slides

Evaluation criteria

Role-appropriate insight. Three distinct uses of evidence: error investigation, recurring patterns and prioritized risks or actions	30%
Search and analytics together. Relevant full-text evidence and correct database aggregations over the stated data scope	25%
Recommendation quality. Specific next steps supported by query evidence; measured facts and hypotheses clearly separated	25%
Execution. A working prototype that runs from a clean checkout, with clear setup instructions and a coherent demonstration, novelty, presentation	20%

Special award: Best MCP Server. Optional and judged separately; it does not change the main score. Judges look at clear tool descriptions, appropriate tool selection by the agent, useful limits on returned results and whether Claude can connect and answer questions using your setup instructions.

Special award: Working with Huge Datasets. Optional and judged separately; it does not change the main score. After completing the core requirements, you may query the listed OpenTelemetry Parquet data or larger Loghub datasets. Show how much data you actually queried, whether the results remain useful and the observed query times and costs, where available. On-demand indexing is an optional extra. Use the available resources; this award does not require the organisers to prepare additional data or infrastructure.

Other bonuses, judged informally: Optional extensions include correlating logs with traces or metrics from the listed multimodal datasets, or using database row-level security to control which records each role can access. Attempt these only after the core requirements are met. They do not replace the required role-specific analysis, and no additional preparation by the organisers is assumed.

Dataset links

Loghub: github.com/logpai/loghub

19 log datasets, about 87 GB in total. Start with two of them.

BGL is your search and trend data. 4.7M lines over 214 days, 320 distinct event templates, five severity levels, structured node IDs and twelve labelled alert categories. Enough text variety that full-text search does real work, and a long enough span for week-over-week analysis.

OpenStack is your aggregation data. 207,820 lines across nova-api, nova-compute and nova-scheduler, with request IDs, tenant IDs, request latency inline and labelled anomalies. This is where counts, latency distributions and error rates come from.

Together that's about 5M lines and 770 MB, loads in minutes. We give you a loader and schema for both, plus a team-ownership table that ties whichever sources you pick into one fictional platform org.

Getting them:

curl -L -o BGL.zip "https://zenodo.org/records/8196385/files/BGL.zip?download=1"
curl -L -o OpenStack.tar.gz "https://zenodo.org/records/8196385/files/OpenStack.tar.gz?download=1"

Keep ?download=1 or you get a preview page instead of the file. -L matters too, Zenodo redirects. Trimming the filename off to browse the folder won't work, Zenodo doesn't list directories.

Before downloading anything, clone the GitHub repo. It has a 2,000-line sample of every dataset plus format docs, one folder each with a *_2k.log inside. You can see the shape of six log formats in two minutes.

All 19:

Dataset	Lines	Size	Span	File
BGL	4,747,963	709 MB	214.7 days	BGL.zip
OpenStack	207,820	58.6 MB		OpenStack.tar.gz
Hadoop	394,308	48.6 MB		Hadoop.zip
HDFS_v1	11,175,629	1.47 GB	38.7 hours	HDFS_v1.zip
HDFS_v2	71,118,073	16.06 GB		HDFS_v2.zip
HDFS_v3 TraceBench	14,778,079	2.96 GB		HDFS_v3_TraceBench.zip
Spark	33,236,604	2.75 GB		Spark.tar.gz
Zookeeper	74,380	9.95 MB	26.7 days	Zookeeper.tar.gz
Thunderbird	211,212,192	29.60 GB	244 days	Thunderbird.tar.gz
HPC	433,489	32.0 MB		HPC.zip
Windows	114,608,388	26.09 GB	226.7 days	Windows.tar.gz
Linux	25,567	2.25 MB	263.9 days	Linux.tar.gz
Mac	117,283	16.09 MB	7 days	Mac.tar.gz
Android_v1	1,555,005	183 MB		Android_v1.zip
Android_v2	30,348,042	3.38 GB		Android_v2.zip
HealthApp	253,395	22.4 MB	10.5 days	HealthApp.tar.gz
Apache	56,481	4.90 MB	263.9 days	Apache.tar.gz
OpenSSH	655,146	70.0 MB	28.4 days	SSH.tar.gz
Proxifier	21,329	2.42 MB		Proxifier.tar.gz

Extensions are inconsistent, some .zip and some .tar.gz, so take the filename from this table rather than guessing. Note OpenSSH is SSH.tar.gz.

A few worth knowing about. **HDFS_v3 (TraceBench)** is instrumented trace data, not plain logs, so it's your option for log-to-trace correlation without leaving Loghub. **OpenSSH** is auth logs — failed logins, brute force attempts, connection patterns — a different flavour of problem and a different kind of insight for the three roles. **Windows and Thunderbird** are the scale options in here at 114M and 211M lines. **Apache and Linux** span 264 days, so long-range trend questions work even though they're small.

Research-use licence, needs a citation: Zhu et al., *Loghub: A Large Collection of System Log Datasets for AI-driven Log Analytics*, ISSRE 2023. For BGL, also Oliner & Stearley, DSN 2007.

Want remote data at scale?

50 billion rows of OpenTelemetry logs sitting in public object storage as Parquet. One day of telemetry from the OpenTelemetry demo app, split into 50 files of a billion rows each, sorted by timestamp.

https://public-pme.s3.eu-west-3.amazonaws.com/text_bench/part_000.parquet

...

https://public-pme.s3.eu-west-3.amazonaws.com/text_bench/part_049.parquet

Standard OTel log record: Timestamp, TraceId, SpanId, TraceFlags, SeverityText, SeverityNumber, ServiceName, Body, plus ResourceAttributes, ScopeAttributes and LogAttributes as maps. Body is the text field you'll be searching.

Don't download it. Point SereneDB at the URL and query it where it is. One file is already a billion rows. Add more if you want a wider window: part_000 alone covers the first 30 minutes of the day, part_000 through part_009 covers the first five hours, all 50 covers the full 24. Reading several at once works, so widening costs you a line of config instead of a loading job. Syntax is at serenedb.com/docs.

Generated from the OpenTelemetry demo app, published by ClickHouse under Apache-2.0.

Want logs, metrics and traces together?

RCAEval (github.com/phamquiluan/RCAEval) has 735 labelled failure cases across Online Boutique, Sock Shop and Train Ticket, with all three signals and eleven fault types.

Nezha has Train Ticket with trace IDs embedded in the application logs, so the log-to-trace join is already in the data.

VIDEO INTRO: Making Sense of Logs - The Right Insights for the Right People - INTRO.mp4